PRODUCT REQUIREMENTS
DOCUMENT

PROOFLINE

Blockchain-powered bug bounty and vulnerability disclosure platform

Project type: Full-stack hybrid Web3 application

Document version: 1.0 — MVP scope

Prepared: 30 October 2026

Intended use: Product planning, implementation guidance, and AI coding-editor context

Product in one sentence
A platform where project owners fund security bounty campaigns, researchers privately submit vulnerability reports, reviewers evaluate them, and a Solidity smart contract handles approved reward payments.

Important product boundary

Proofline does not automatically detect vulnerabilities or decide whether a report is valid. Human reviewers make that decision. Blockchain provides transparent campaign funding and rule-enforced settlement; private report content remains off-chain.

Document map

No.	Section
01	Executive summary and product goals
02	Problem, users, and product workflow
03	Scope: MVP features and exclusions
04	Functional requirements
05	On-chain vs off-chain data design
06	Technical architecture and technology stack
07	Smart-contract requirements
08	Backend, database, and API requirements
09	Frontend screens and user experience
10	Security, privacy, and reliability
11	Testing and acceptance criteria
12	Implementation phases and deliverables
13	Risks, assumptions, and decisions
14	AI coding-editor implementation brief

This document defines a realistic learning MVP. Build the smallest complete workflow first; do not start with advanced decentralization, tokenomics, or multi-chain features.

01. Executive summary and product goals

Proofline is a hybrid Web3 bug bounty platform. A project owner creates a campaign and deposits ETH into a smart-contract escrow. A security researcher submits a vulnerability report privately. An authorized reviewer evaluates the report. If approved, the smart contract pays the researcher according to the campaign's published rules.

Primary goals

- Demonstrate practical Solidity development using a real application workflow.
- Show how a React frontend interacts with Ethereum smart contracts through ethers.js.
- Use Hardhat because the developer is strongest in JavaScript and wants a JavaScript/TypeScript-friendly development, testing, and deployment workflow.
- Demonstrate a clear separation between blockchain state and conventional application data.
- Build an interview-ready project with meaningful tests, documented security assumptions, a deployed testnet contract, and a working demo.

Success definition

- An owner can create a campaign and successfully deposit ETH.
- A researcher can submit a private report linked to a campaign.
- A reviewer can approve or reject a report according to defined permissions.
- An approved report can be paid exactly once by the smart contract.
- The UI clearly displays transaction states and the blockchain transaction reference.
- The repository includes reproducible setup instructions, tests, architecture documentation, and a short demo.

Non-goal

The MVP is not a production-grade bug bounty marketplace and does not guarantee that submitted reports are safe, correct, unique, or legally authorized. It is a learning and portfolio project using testnet funds only.

02. Problem, users, and product workflow

Problem statement

Website and software owners need a structured way to invite security researchers to find vulnerabilities, receive reports, review findings, and pay rewards. In an ordinary application, payment promises and payment records are controlled entirely by the platform operator. Proofline uses a smart contract to hold campaign funds and enforce the payment rules after an authorized approval.

User roles

Role	Who they are	Main responsibilities
Campaign owner	The website or project owner who wants security testing.	Create campaign, define scope and reward rules, fund campaign, review campaign status, request eligible refund after closure.
Researcher	A security tester who looks for vulnerabilities.	Browse campaigns, submit a private report, provide evidence, view report status, receive an approved reward.
Reviewer	An authorized person appointed to evaluate reports.	Access permitted reports, assess validity and severity, add review notes, approve or reject, initiate dispute workflow if enabled.

Example end-to-end scenario

- A store owner creates a ShopKart campaign offering a reward of 0.02 ETH for a qualifying vulnerability and deposits 0.1 ETH into the campaign contract.
- A researcher discovers an access-control issue and submits a detailed report and evidence through the private application interface.
- The backend stores the report securely and records a commitment/hash on-chain. The report text and screenshots are not published to Ethereum.
- A reviewer checks the report and approves it. The approval action is restricted to an authorized reviewer or campaign authority, as defined by the MVP permissions.
- The contract transfers 0.02 ETH to the researcher's wallet and emits a payment event. The application updates the report and campaign views from the confirmed transaction/event.

Key terminology

Term	Meaning in Proofline
Bounty	The reward offered for a valid vulnerability report.
Bounty fund / escrow	ETH deposited into the smart contract and reserved for campaign rewards.
On-chain	Data and actions recorded or enforced by the Ethereum smart contract.
Off-chain	Application data stored outside Ethereum, such as MongoDB records and private files.
Commitment/hash	A cryptographic fingerprint used to later check that data matches a previously committed value. A hash is not encryption.
Testnet	A blockchain network used for development and testing with test ETH, not real money.

03. Scope: MVP features and exclusions

In-scope MVP

Feature area	MVP scope
Wallet connection	Connect an EVM wallet, display active address and network, prompt the user to switch to the configured testnet.
Campaign management	Create campaign, publish title/description/scope/rules/deadline/reward terms, fund campaign, view campaign details.
Campaign discovery	List and filter active campaigns using off-chain metadata; show authoritative funding/status values from the contract.
Private report submission	Researcher submits title, description, severity suggestion, reproduction steps, and optional evidence files through authenticated backend routes.
Review workflow	Reviewer views assigned/permitted reports, adds private notes, approves or rejects with a reason.
Reward settlement	Approved report triggers contract payout to the researcher's wallet, subject to campaign balance and contract rules.
Refund/closure	Owner can close a campaign and request a refund only under clearly defined contract rules; no refund of funds already paid or reserved.
Activity history	Display campaign creation/funding, report status, approval, payout, and refund history.
Admin controls	Basic role assignment and reviewer management for the project administrator.

Explicitly out of scope for version 1

- Multi-chain deployment, cross-chain bridges, or support for multiple currencies.
- DAO governance, governance tokens, staking, tokenomics, or NFT rewards.
- AI vulnerability detection, automatic severity scoring, or automated approval.
- Fully decentralized arbitration or trustless determination that a vulnerability is genuine.
- Public disclosure of report contents or public file hosting.
- Complex enterprise integrations, paid subscriptions, fiat payments, and production customer onboarding.
- Mobile applications and browser extensions.

MVP simplification

For the first release, use one campaign owner, one designated reviewer (or a small allowlist), one EVM testnet, ETH rewards, and a straightforward approval/rejection process. Add independent dispute handling only after the basic lifecycle is tested and stable.

04. Functional requirements

FR-01: Wallet and network

- The user can connect and disconnect an EVM wallet through the frontend.
- The application displays the connected wallet address and current network.
- Transactions are disabled or clearly blocked when the user is on an unsupported network.
- The application never asks users to share seed phrases or private keys.

FR-02: Campaign creation and funding

- Owner enters campaign name, project URL, description, in-scope assets, out-of-scope assets, rules, start/end time, reward amount, and maximum campaign budget.
- The UI shows the ETH deposit amount and estimated transaction information before wallet confirmation.
- Campaign creation/funding is submitted to the smart contract and only shown as successful after transaction confirmation.
- The application stores supporting display metadata in MongoDB and links it to the on-chain campaign ID.

FR-03: Report submission

- Researcher selects an active campaign and submits report details through the backend.
- The backend validates fields, file types, file sizes, authorization, and campaign eligibility.
- Report details and evidence are private by default and accessible only to authorized parties.
- The backend creates a commitment using a documented canonical format and a unique random nonce; only the commitment and minimum required identifiers are sent on-chain if the design requires it.
- The researcher receives a submission reference and can track status.

FR-04: Review and decision

- Only authorized reviewers can access and decide reports.
- Reviewer can mark a report approved or rejected and must provide a decision reason.
- The backend records detailed review notes privately.
- An approval action invokes the contract payout flow or authorizes a separate payout transaction according to the final contract design.
- Rejected reports cannot be paid unless a later appeal/review changes the outcome through an explicitly permitted workflow.

FR-05: Payout, closure, and history

- A report cannot be paid more than once.
- A payout cannot exceed the campaign's available funds or the reward amount allowed by campaign rules.
- The payout recipient must be the wallet address bound to the submission, subject to the final contract design.
- Campaign closure and refunds must respect deadlines, outstanding approved rewards, and contract accounting.
- The UI displays transaction hash, network, amount, recipient, and confirmation status for on-chain actions.

05. On-chain vs off-chain data design

Design principle

Put data on Ethereum only when the contract needs to enforce a rule or when a public, tamper-resistant record is valuable. Store private, large, frequently edited, or personally identifiable information off-chain. All blockchain data should be treated as public.

Data	Storage	Reason
Campaign ID, owner wallet, campaign state	On-chain	Needed to identify and enforce campaign ownership/state.
Campaign funding and available balance	On-chain	Contract must control and account for ETH.
Reward terms needed for settlement	On-chain	Payout rules must be enforced by contract.
Submission commitment/hash and minimal status	On-chain (if required)	Can provide a verifiable reference without publishing report content.
Approval and payout events	On-chain	Creates an auditable record of settlement.
User name, email, profile, preferences	Off-chain MongoDB	Personal and editable application data.
Full report, reproduction steps, private comments	Off-chain MongoDB	Confidential information that should not be public.
Screenshots, videos, attachments	Private object storage	Large files are unsuitable for Ethereum storage.
Notifications, search indexes, dashboard summaries	Off-chain MongoDB	Application convenience and fast querying.

What does off-chain storage mean?

Off-chain simply means stored outside the blockchain. For this project, use MongoDB for structured application records and private file storage for uploaded evidence. During local development, files may be stored in a protected local uploads directory. For a deployed environment, use a private Amazon S3 bucket or equivalent object storage. Store only file metadata and storage keys in MongoDB, not public file URLs.

Recommended beginner setup

- MongoDB: users, campaigns' descriptive metadata, report metadata, review notes, role assignments, and notification records.
- Local file storage for development: private files outside the publicly served frontend directory, with backend authorization checks.
- Amazon S3 (later deployment): private bucket, server-side encryption, restricted IAM permissions, short-lived signed download URLs, and upload validation.
- Ethereum: campaign escrow, key payout terms, payment/refund logic, and relevant events.

Hash/commitment handling

A report hash can help prove that a later-presented report matches a previously committed report. It does not hide the report. Use a cryptographically secure random nonce and a documented canonical serialization of the report. Keep the nonce protected until disclosure is appropriate. Do not hash only a predictable title or short text. Avoid placing personal data, report text, attachment URLs, or secrets in transaction input or events.

06. Technical architecture and technology stack

Layer	Technology	Responsibility
Frontend	React + TypeScript + Tailwind CSS	User dashboards, forms, wallet connection, report workflows, transaction status.
Web3 client	ethers.js	Connect to wallet provider, read contract state, prepare and submit transactions, listen to events.
Smart contracts	Solidity	Campaign creation, ETH escrow, permissions, payout, refund and event emission.
Blockchain development	Hardhat	Compile, local network, deploy scripts, automated tests, contract verification workflow.
Backend API	Node.js + Express	Authentication, authorization, validation, private report workflows, file access, indexing.
Database	MongoDB + Mongoose	Application metadata, user records, report records, reviewer notes, activity projections.
File storage	Local private storage initially; S3 later	Evidence attachments and report files.
Network	Hardhat local network; Sepolia testnet for demo	Local development and public testnet demonstration.
Version control / CI	Git + GitHub Actions (recommended)	Code history, automated linting, tests and build checks.

Why Hardhat?

The developer's existing strength is JavaScript. Hardhat is therefore the selected Ethereum development framework because it supports a JavaScript/TypeScript-oriented workflow for compiling, testing, running a local development network, scripting deployments, and integrating contract tests into the same engineering workflow as the React/Node.js application. Solidity remains the smart-contract language; Hardhat is the development and deployment framework, not a replacement for Solidity.

High-level architecture

```
React frontend → ethers.js + wallet provider → Solidity contract deployed to Ethereum testnet.
React frontend → Node.js/Express API → MongoDB.
Node.js/Express API → private file storage for report evidence.
Backend event indexer/listener → reads contract events and updates off-chain activity projections.
Contract state remains authoritative for funds and payout rules.
```

Source of truth rules

- Smart contract is the source of truth for escrow balances, on-chain campaign state, and whether a payout transaction succeeded.
- MongoDB is the source of truth for private report content, user profile data, reviewer notes, and application-only metadata.
- The backend must not fabricate a successful payout based on a client-side message. Verify the transaction receipt and/or indexed contract event.
- The frontend must not trust user-submitted role values. Backend authorization and contract permissions must be independently enforced.

07. Smart-contract requirements

Contract design objective

Implement the smallest understandable contract that can create a funded campaign and pay an approved researcher exactly once. Keep the contract focused on money and enforceable state. Do not put the full report review system, user profiles, or file handling into Solidity.

Suggested first contract

BountyEscrow.sol — initial responsibilities

- Create a campaign with owner, reward amount, campaign deadline, and any required funding parameters.
- Accept ETH deposits and track campaign accounting.
- Record a minimal submission identifier/commitment if the selected flow requires on-chain commitments.
- Restrict approval/payout authorization to the campaign owner or explicitly authorized reviewer role.
- Pay the designated researcher only when the campaign and submission satisfy contract rules.
- Prevent duplicate payouts and invalid state transitions.
- Support campaign closure and refund only under clearly specified conditions.
- Emit events for campaign creation, funding, submission commitment, approval, rejection (if stored on-chain), payout, closure, and refund.

Illustrative state model

Entity	Possible states	Notes
Campaign	Active → Closed	A campaign may stop accepting new reports after its deadline or owner closure, subject to contract rules.
Submission	Submitted → Approved → Paid	Alternative terminal path: Submitted → Rejected. Do not allow Paid twice.
Refund	Eligible → Refunded	Only after campaign closure and after accounting for outstanding liabilities.

Core contract rules / invariants

- Only the campaign owner can create or manage their campaign, except for explicitly assigned reviewer permissions.
- Only an authorized reviewer/owner can approve a submission for payout.
- A submission identifier can be settled no more than once.
- Total payouts plus refunds must never exceed ETH deposited into the campaign.
- Payout amount must follow the campaign's stored reward rules and available balance.
- A campaign cannot be refunded while approved but unpaid rewards remain outstanding.
- Use checks-effects-interactions and/or OpenZeppelin protections where appropriate to reduce reentrancy risk.
- Handle failed ETH transfers safely; consider a pull-payment/withdrawal pattern if it simplifies security.
- Use custom errors, events, and clear NatSpec documentation.

Important design choice for the MVP

Keep the reviewer workflow simple. A reviewer decision can be recorded off-chain, but a payout must still require an authorized on-chain action. For a first version, use the campaign owner as the only payout approver; add a reviewer allowlist after the basic flow works. This avoids prematurely building complex on-chain reviewer governance.

Do not assume the contract can verify the truth of a report. It can verify permissions, balances, deadlines, and whether a submission has already been paid. Human judgment remains outside the contract.

08. Backend, database, and API requirements

Backend responsibilities

- Authenticate application users and bind accounts to wallet addresses where required.
- Enforce role-based access on every protected endpoint.
- Validate and store campaign metadata and private report content.
- Handle secure evidence upload and authorized download.
- Generate report commitments using a consistent, versioned format.
- Listen for contract events or query receipts to build application activity history.
- Reconcile off-chain projections with on-chain state; handle duplicate events and retries idempotently.
- Never store wallet private keys or seed phrases.

Suggested MongoDB collections

Collection	Example fields (illustrative)
users	displayName, email, passwordHash or authProviderId, walletAddress, roles, createdAt
campaigns	chainId, contractAddress, onChainCampaignId, ownerWallet, title, description, projectUrl, scope, rules, deadline, statusMetadata
reports	campaignId, researcherUserId, researcherWallet, title, privateDescription, severity, encryptedOrProtectedEvidenceRefs, commitment, status, createdAt
reviews	reportId, reviewerUserId, decision, privateNotes, reason, decidedAt
chainTransactions	chainId, txHash, transactionType, relatedCampaignId, relatedReportId, status, blockNumber
notifications	userId, type, message, readAt, createdAt

Data modeling notes

- Use the combination of chainId, contractAddress, and on-chain campaign ID as a robust campaign reference.
- Store wallet addresses normalized to a consistent checksum/lowercase comparison strategy.
- Do not use MongoDB document IDs as proof of on-chain identity.
- Avoid duplicating on-chain balances as editable database fields. Cache them only as projections with a clear refresh/reconciliation mechanism.
- Store report evidence using private storage keys, not publicly accessible URLs.

Suggested API surface

Method + route	Purpose / access
POST /api/auth/register	Create an application account (if email/password auth is used).
GET /api/campaigns	List campaigns and descriptive metadata.

Method + route	Purpose / access
GET /api/campaigns/:id	Campaign detail; combine MongoDB metadata with contract reads.
POST /api/campaigns/:id/reports	Researcher submits a private report and evidence.
GET /api/reports/mine	Researcher views their own reports.
GET /api/reviewer/reports	Authorized reviewer lists assigned/permitted reports.
GET /api/reviewer/reports/:id	Authorized reviewer reads a report.
POST /api/reviewer/reports/:id/decision	Record reviewer decision and trigger/prepare settlement flow.
GET /api/activity	Return indexed activity for the authenticated user.
POST /api/webhooks/chain-events (internal)	Optional internal event ingestion endpoint; protect from public spoofing.

09. Frontend screens and user experience

Screen	Required UI elements
Landing page	Product explanation, how it works, active campaign preview, connect wallet.
Campaign explorer	Campaign list, search/filter, status, reward amount, deadline, funded amount.
Create campaign	Campaign metadata form, reward/deposit preview, wallet transaction confirmation.
Campaign detail	Scope, rules, deadline, owner wallet, on-chain funding/status, submit report action.
Researcher dashboard	My submissions, report status, reviewer decision, payout status and transaction link.
Submit report	Private report form, reproduction steps, severity, evidence upload, privacy notice.
Reviewer dashboard	Authorized report queue, report detail, evidence viewer, private notes, approve/reject controls.
Owner dashboard	My campaigns, funding status, reports summary, closure/refund eligibility and activity.
Transaction detail/status	Pending, confirmed, failed/reverted; transaction hash, network, amount and recipient.

UX requirements

- Clearly distinguish an application action from a blockchain transaction. For example, 'Report uploaded' is not the same as 'Payout confirmed'.
- Show wallet confirmation, pending, confirmed, rejected by wallet, and reverted/error states.
- Never display 'Paid' until the contract transaction is confirmed and validated.
- Explain that testnet ETH has no intended real-world value.
- Make private report visibility explicit: researcher and authorized reviewers only.
- Show a concise warning before a user submits sensitive security details.

10. Security, privacy, and reliability

Smart-contract security

- Use OpenZeppelin components where appropriate; do not invent custom cryptography or access-control primitives.
- Test unauthorized access, reentrancy attempts, duplicate settlement, insufficient funds, deadline boundaries, failed transfers, and invalid state transitions.
- Use Solidity version pinning and a reproducible dependency lockfile.
- Do not deploy contracts holding real funds for the portfolio MVP.
- Run static analysis or a suitable security tool as an additional check; automated tools do not prove a contract is secure.

Application and data security

- Use server-side authorization on every report, reviewer, and admin endpoint.
- Hash passwords with a suitable password hashing algorithm if password authentication is implemented; never store plaintext passwords.
- Validate upload size, extension, MIME type, and file contents; consider malware scanning before production use.

- Keep evidence storage private. Use short-lived authorized download links or backend streaming after permission checks.
- Encrypt sensitive report data at rest where feasible and use HTTPS in deployment.
- Keep secrets in environment variables or a secrets manager; never commit RPC keys, database credentials, JWT secrets, or private keys.
- Do not log report contents, passwords, access tokens, or sensitive evidence.
- Use rate limits and request validation on authentication, upload, and report submission endpoints.

Reliability and consistency

- A wallet transaction may be pending, dropped, replaced, reverted, or confirmed. Model these states explicitly.
- Backend event processing must be idempotent so the same blockchain event does not create duplicate records.
- Use transaction receipts and contract reads to reconcile database projections.
- Design for a user closing the browser after sending a transaction; the app should recover status from the transaction hash or indexed events.

11. Testing and acceptance criteria

Smart-contract test cases

Test	Expected result
Create campaign with valid parameters	Campaign is created and event emitted.
Create campaign with invalid reward/deadline	Transaction reverts with a clear error.
Fund campaign	Contract balance and campaign accounting increase correctly.
Non-owner attempts owner-only action	Transaction reverts.
Unauthorized reviewer attempts approval	Transaction reverts.
Approve valid submission	Submission becomes eligible for settlement according to contract rules.
Pay approved submission	Correct amount reaches the intended wallet and paid state is recorded.
Pay the same submission twice	Second payout reverts.
Attempt payout with insufficient funds	Transaction reverts without corrupting accounting.
Refund before eligible closure	Transaction reverts.
Refund after valid closure	Only eligible remaining funds are returned.
Malicious/reentrant receiver test	Contract does not allow unauthorized repeated withdrawals or accounting corruption.

Application acceptance criteria

- A user can connect a wallet and see the correct network/address.
- A campaign can be created and funded from the UI, with a confirmed transaction hash.
- Campaign details show contract-backed funding and status.
- A researcher can submit a report and later retrieve it when authenticated.
- A user cannot read another researcher's private report unless explicitly authorized.
- Only an authorized reviewer can decide a report.

- A successful payout is reflected in the UI only after confirmation.
- The activity view can show the relevant transaction reference.
- The README allows another developer to install, test, deploy locally, and run the app using documented steps.

Minimum quality gates

- Contract unit tests pass on the local Hardhat network.
- Frontend production build succeeds.
- Backend validation and authorization tests cover protected routes.
- No secrets or private keys are committed.
- At least one end-to-end demo flow works from campaign creation through researcher payout on a local network or testnet.
- Known limitations and security assumptions are documented.

12. Implementation phases and deliverables

Build in vertical slices. Each phase should produce something runnable and testable.

Phase	Focus	Deliverable / exit criteria
0. Setup	Repository, Hardhat, React, backend, MongoDB, environment configuration.	Project starts locally; Hardhat compiles; frontend and API health checks work.
1. Contract foundation	Campaign creation, funding, basic owner permissions, events.	Contract tests pass; deploy to local Hardhat network.
2. Payout lifecycle	Submission identifier, approval authorization, one-time payout, closure/refund rules.	Tests cover valid payout, duplicate payout, unauthorized calls, insufficient balance and refund conditions.
3. Frontend Web3	Wallet connection, create/fund campaign, campaign detail, transaction status.	User can create and fund a campaign from React.
4. Backend and reports	Auth, roles, MongoDB, private report submission, file handling.	Researcher submits report; authorized reviewer can retrieve it.
5. Review and settlement	Reviewer decision, contract payout action, event indexing and UI history.	Complete end-to-end lifecycle works.
6. Hardening and demo	Security review, CI, documentation, Sepolia deployment, demo recording.	Public testnet demo, verified source where available, README, architecture and threat model.

Recommended order for a JavaScript-first developer

- Start with Solidity contract functions and Hardhat tests before building all UI screens.
- Use JavaScript/TypeScript for Hardhat scripts, backend, frontend, and tests where supported.
- Deploy locally first. Do not depend on a public testnet while basic contract behavior is still changing.
- Integrate ethers.js only after the contract ABI and local deployment are stable.
- Add MongoDB and report uploads after the on-chain deposit/payout flow works.
- Deploy to Sepolia after local tests and a manual security review.

Suggested repository structure

```
proofline/
■■■ contracts/ # Solidity contracts
■■■ scripts/ # Hardhat deployment and utility scripts
■■■ test/ # Contract unit/integration tests
■■■ frontend/ # React + TypeScript + Tailwind
■■■ backend/ # Node.js + Express API
■■■ docs/ # PRD, architecture, threat model, demo notes
■■■ hardhat.config.ts
■■■ package.json
■■■ README.md
```

13. Risks, assumptions, and decisions

Risk / assumption	Mitigation / decision
Reviewer may be dishonest or mistaken.	The platform records who made the decision; the contract does not claim to prove vulnerability validity. Add dispute handling only after MVP.
Sensitive report details could leak.	Keep full reports and evidence off-chain, enforce role access, use private storage and avoid sensitive on-chain data.
Smart contract bug could lock or lose test funds.	Use testnet only, write adversarial tests, review accounting and access control, and do not use real funds.

Risk / assumption	Mitigation / decision
Database and chain may disagree temporarily.	Treat chain as authoritative for funds and payout; index events idempotently and reconcile.
Owner may fund less than expected or campaign may run out of funds.	Show available balance, enforce payout limits, and define clear campaign funding rules.
Gas costs or network issues affect UX.	Show pending/error states and allow users to retry safely without duplicating settlement.
MVP scope becomes too large.	Defer DAO, multi-chain, AI scoring, complex arbitration, subscriptions, and token features.

Open decisions to finalize before coding

- Will the owner alone approve reports in MVP, or will there be a designated reviewer wallet? Recommended: owner-only payout approval initially, with reviewer decision recorded off-chain.
- Will each campaign offer one fixed reward amount, or reward tiers by severity? Recommended: one fixed reward amount for the first version.
- Will the contract hold one campaign's funds in a shared contract balance with per-campaign accounting, or use a separate escrow per campaign? Recommended: one contract with carefully tested per-campaign accounting for learning simplicity.
- Will report commitments be recorded on-chain in the first iteration? Recommended: add this after basic escrow/payout works; do not let it delay the first vertical slice.
- What exact refund conditions apply to unused campaign funds? Define before implementing withdrawals.

Assumptions

- This is an educational portfolio project, not a production security disclosure service.
- Use Sepolia testnet and test ETH for public demonstration.
- Use a single EVM-compatible network and ETH-denominated rewards in version 1.
- Use wallet signatures for blockchain transactions; application login may be separate but should be linked to a wallet through a signed challenge if wallet ownership must be proven.
- The developer has existing JavaScript, React, Node.js, MongoDB, Solidity, ethers.js, and introductory Hardhat knowledge.

14. AI coding-editor implementation brief

Copy the following brief into your AI coding editor as project context. Ask the editor to implement one phase at a time; do not ask it to generate the entire application in one pass.

PROJECT: PROOFLINE

Build a beginner-friendly but well-engineered hybrid Web3 bug bounty platform. The application has three roles: Campaign Owner, Researcher, and Reviewer. Owners create campaigns and deposit ETH into a Solidity escrow contract. Researchers submit private vulnerability reports through the backend. Reviewers inspect reports and approve or reject them. An approved reward is paid by the smart contract to the researcher's wallet, exactly once.

STACK
- Smart contracts: Solidity.
- Ethereum development framework: Hardhat, selected because the developer is strongest in JavaScript and wants JS/TS-based scripts and tests.
- Frontend: React, TypeScript, Tailwind CSS, ethers.js.
- Backend: Node.js, Express.
- Database: MongoDB with Mongoose.
- File storage: private local storage for development; private S3-compatible object storage later.
- Network: Hardhat local network first; Sepolia testnet for demo.

ARCHITECTURE RULES
1. The smart contract is authoritative for campaign escrow, on-chain state, payout eligibility rules, and completed payments.
2. MongoDB stores user profiles, campaign descriptions, report content/metadata, reviewer notes, and application activity projections.
3. Full vulnerability reports, screenshots, evidence, emails, names, and other private data must never be stored on-chain or emitted in contract events.
4. A hash/commitment is not encryption. If commitments are implemented, use a unique cryptographically secure nonce and a documented canonical format.
5. Do not store private keys or seed phrases. Never trust client-provided role values. Enforce backend and contract permissions independently.
6. Never show a payout as successful until the transaction receipt/event is confirmed and validated.

MVP FEATURES
- Wallet connection and network validation.
- Owner creates and funds a campaign.
- Campaign explorer and campaign detail page.
- Researcher submits private report and evidence.
- Reviewer securely views report and approves/rejects with reason.
- Authorized on-chain payout to researcher exactly once.
- Campaign closure and safe refund of eligible unused funds.
- Transaction history and clear pending/confirmed/failed states.

IMPLEMENTATION RULES
- Work phase by phase: setup, contract foundation, payout lifecycle, frontend Web3, backend reports, review/settlement, hardening/demo.
- Before coding, propose a concise implementation plan and list files to create or modify.
- Do not invent additional features outside the MVP without asking.
- Write contract tests before integrating the frontend.
- Include validation, error handling, loading states, and meaningful comments for learning.
- Use established libraries such as OpenZeppelin where appropriate; do not write custom cryptography.
- Include tests for unauthorized access, duplicate payouts, insufficient funds, deadline boundaries, refund rules, and reentrancy attempts.
- Keep environment variables in .env files excluded from Git and provide .env.example with placeholders.
- Provide setup/run/test/deploy instructions and explain important decisions in plain language.

FIRST TASK
Do not build the full application yet. Start by designing the BountyEscrow.sol contract interface, campaign data structure, events, state transitions, and test cases. Explain the design in beginner-friendly language. Wait for approval before implementing the next phase.

Final guidance

Treat this PRD as the MVP baseline. The best first milestone is not a beautiful dashboard; it is a tested contract that accepts a campaign deposit and safely pays an approved researcher once. Build the application around that working core.

End of document